

Full-Stack .NET with Angular and ASP.NET Core

Full Stack .NET Development means building both the front-end and back-end of a web application using Microsoft's .NET tools. It allows developers to create everything from the user interface to the server and database using one technology stack. This makes development faster and easier to manage. It is popular because it works well for creating secure and powerful websites and apps that can run on different platforms like Windows and Linux.

New Batch: 15th September 2025, 8:30 PM to 10:00 PM, IST

Demo Sessions: 15/16/17 September 2025, 8:30 PM to 10:00 PM, IST

Join Demo Sessions using the below Zoom Meeting Link:

<https://us06web.zoom.us/j/85482461029?pwd=IdfioAqnHLsPsiGv15o6KND1mckx6t.1>

Meeting ID: 854 8246 1029

Passcode: 345008

Contact Us:

 **Course Duration:** 4 Months (Monday to Friday), Daily 1:30 Hours

 **Course Registration Link:** <https://forms.gle/28KZqPmE37cpqeru7>

 **Join our Telegram Group for more information:** <https://telegram.me/dotnettutorials>

 **Mobile / WhatsApp Number:** +91 70218 01173

 **Course Details:** <https://dotnettutorials.net/lesson/full-stack-net-with-angular-and-asp-net-core-online-training/>

ASP.NET Core Web API Syllabus

ASP.NET Core Web API is a Framework from Microsoft used to build web services that send and receive data over the internet. These services let different apps, like websites or mobile apps, talk to each other. ASP.NET Core Web API is fast, lightweight, and can run on many operating systems. It helps developers create APIs that are easy to maintain and secure.

Chapter 1: Introduction and Environment Setup for ASP.NET Core Web API

Gain a deep understanding of Web API fundamentals, the strengths of ASP.NET Core for API development, and how to efficiently set up and configure your development environment. By the end, you will be able to create, structure, and launch a new ASP.NET Core Web API project with proper environment settings and hosting configurations.

- **What is a Web API?** Understand the concept and differences between REST, SOAP, and GraphQL.
- **Why use ASP.NET Core for Web APIs?** Learn the benefits and common use cases of ASP.NET Core.
- **Overview of ASP.NET Core and Web API architecture.** See how ASP.NET Core enables scalable and performant APIs.
- **Installing .NET SDK and IDEs.** Set up Visual Studio or VS Code for ASP.NET Core development.
- **Creating a new ASP.NET Core Web API project.** Follow step-by-step instructions for project setup.
- **Understanding project folder and file structure.** Learn the role of each important file and folder.
- **Importance of Program.cs.** Discover how application startup is configured in ASP.NET Core.
- **Running and testing the API.** Use tools like Postman, Swagger, and Fiddler for API testing.
- **Basic configuration of launchSettings.json.** Manage different environments like development and production.
- **Overview of hosting models.** Learn in-process vs out-of-process hosting in ASP.NET Core.

 **Have Questions? Contact Us on WhatsApp: +91 70218 01173 | Telegram: @dotnettutorials**

Chapter 2: Routing in ASP.NET Core Web API

Learn how ASP.NET Core routes HTTP requests to controller actions. Learn to define, customize, and troubleshoot routes, handle route parameters and constraints, and apply advanced routing techniques to control API endpoint behaviour precisely.

- **What is routing?** Understand how HTTP requests are directed to controller actions.
- **Types of routing.** Compare attribute routing and conventional routing, including pros and cons.
- **Defining routes in controllers and actions.** Learn when and how to use route attributes.
- **Route parameters and constraints.** Capture and validate data passed in routes.
- **Default values and optional parameters.** Discover how and why to use these features.
- **Customizing routes.** Use advanced route templates and patterns for flexibility.
- **Route precedence and ordering.** Understand how ASP.NET Core selects the correct route.

Chapter 3: HTTP Methods in ASP.NET Core Web API

Understand how HTTP verbs are mapped to API endpoints in ASP.NET Core. You'll master when and how to use GET, POST, PUT, DELETE, PATCH, and other HTTP methods, and apply best practices for RESTful endpoint design, method security, and supporting CORS.

- **Overview of HTTP verbs.** Know the purpose of GET, POST, PUT, DELETE, PATCH, etc.
- **Mapping HTTP methods to actions.** Use attributes like [HttpGet] to bind methods.
- **Idempotent vs non-idempotent operations.** Learn why this distinction matters for API design.
- **Designing RESTful endpoints.** Follow best practices for resource naming and method usage.
- **Safe and unsafe HTTP methods.** Understand implications for caching and proxies.
- **Using HTTP OPTIONS method.** Support CORS preflight requests properly.
- **HTTP method restrictions.** Secure your API by allowing only intended verbs.

Chapter 4: Return Types and HTTP Status Codes in ASP.NET Core Web API

Master the use of action return types, how to return appropriate HTTP status codes, customize responses including headers and content negotiation, and structure responses for large datasets with pagination and metadata.

- **Action result types.** Understand IActionResult, ActionResult<T>, and concrete return types.
- **Returning HTTP status codes.** Use Ok, NotFound, BadRequest, NoContent, and others properly.
- **Customizing responses.** Use ObjectResult for flexible response structures.
- **ProblemDetails and error standards.** Implement standardized error responses.
- **Using ProducesResponseType attribute.** Document your API responses clearly.
- **Content negotiation.** Return XML or JSON depending on client requests.
- **Handling file and binary responses.** Serve files and streams effectively.
- **Custom response headers.** Manage HTTP headers to enhance responses.

Chapter 5: Model Binding in ASP.NET Core Web API

Learn how ASP.NET Core automatically binds HTTP request data to action parameters, explore various binding sources, handle simple and complex types, create custom model binders, and manage file uploads with multipart forms.

- **What is model binding?** Understand how HTTP data is converted to action parameters.
- **How model binding works.** Learn about binding sources and their priority order.
- **Binding from route, query, form, headers, body.** Know when to use each source.



- **Binding simple vs complex types.** Handle different data structures efficiently.
- **Using binding attributes.** Apply [FromBody], [FromRoute], [FromQuery] correctly.
- **Binding collections and nested objects.** Manage complex nested data inputs.
- **Custom model binders.** Extend default behavior with custom binders.
- **Model binding for file uploads.** Process multipart/form-data file inputs.

Chapter 6: Validations in ASP.NET Core Web API

Understand the importance of validating API input, use built-in and custom validation techniques, handle validation errors gracefully, integrate Fluent Validation, and coordinate client- and server-side validations for complex models.

- **Why Validation Matters.** Ensure data integrity and provide user feedback.
- **Data Annotations and Built-in Validators.** Use standard validation attributes.
- **Custom Validation Attributes.** Create your own validation logic when needed.
- **Model State and Error Handling.** Detect validation errors and respond properly.
- **Fluent Validation Integration.** Use fluent syntax for complex validation.
- **Returning Validation Errors.** Communicate errors clearly to API clients.
- **Client-side vs Server-Side Validation.** Understand coordination between them.
- **Validating Nested and Collection Objects.** Handle validations on complex models.

Chapter 7: Dependency Injection in ASP.NET Core Web API

Develop a solid grasp of dependency injection principles, including service registration, lifetimes, and usage of the Options pattern. Learn to leverage DI for loosely-coupled, maintainable, and easily testable code across your API.

- **What is Dependency Injection?** Understand its benefits for maintainability and testing.
- **Service Registration in DI Container.** Know AddTransient, AddScoped, AddSingleton lifetimes.
- **Constructor Injection.** Inject dependencies into controllers and services.
- **Service Lifetimes and Scopes.** Choose the appropriate lifetime for your services.
- **Using IServiceProvider.** Advanced scenarios for service resolution.
- **Options Pattern.** Manage configurations with strongly typed classes.

Chapter 8: EF Core and Database Integration in ASP.NET Core Web API

Explore Entity Framework Core fundamentals, configure database providers, create and manage entity models and DbContext, perform CRUD operations, optimize performance, handle concurrency, use transactions, and apply logging for diagnostics.

- **What is EF Core and why use it?** Learn the benefits of ORM for data access.
- **Supported database providers.** Understand SQL Server, SQLite, and InMemory options.
- **Code First vs Database First.** Choose the right approach for your project.
- **Defining entity models and DbContext.** Create your data model and context class.
- **Connection strings and migrations.** Manage schema evolution effectively.
- **CRUD operations.** Implement Create, Read, Update, Delete with EF Core.
- **Entity relationships.** Understand one-to-one, one-to-many, many-to-many.
- **Querying with LINQ.** Write expressive database queries.
- **Tracking vs No-tracking queries.** Improve performance using no-tracking.
- **Seeding data.** Initialize your database with default data.
- **Using transactions.** Ensure data consistency in operations.
- **EF Core logging and diagnostics.** Track and troubleshoot performance issues.
- **Database concurrency handling.** Manage optimistic and pessimistic concurrency.
- **Optimizing EF Core performance.** Use no-tracking, compiled queries, and batching.



Chapter 9: Data Transfer Objects and AutoMapper in ASP.NET Core Web API

Understand the role of DTOs in separating API contracts from data models, create and manage DTOs, automate mapping between entities and DTOs using AutoMapper, and optimize mapping for performance and complexity.

- **What are DTOs and why use them?** Decouple API contracts from database entities.
- **Creating and using DTO classes.** Structure DTOs for clear data transfer.
- **Mapping between entities and DTOs.** Learn manual and automated mapping approaches.
- **Introduction to AutoMapper.** Automate object-to-object mapping easily.
- **Configuring mapping profiles.** Centralize and organize mapping logic.
- **Mapping collections and nested objects.** Handle complex DTOs with nested data.

Chapter 10: Async Programming in ASP.NET Core Web API

Learn asynchronous programming concepts, implement async methods in controllers and data access layers, manage cancellation tokens, and optimize API responsiveness and scalability through efficient async coding practices.

- **Synchronous vs Asynchronous Programming.** Understand concepts and trade-offs.
- **Why async matters in Web APIs.** Improve scalability and responsiveness.
- **Using async/await.** Implement asynchronous methods properly.
- **Asynchronous EF Core Queries.** Query databases without blocking threads.
- **Performance Benefits.** See real-world impact on API throughput.
- **Performance Optimization.** Use ValueTask vs Task for efficiency.

Chapter 11: Middleware in ASP.NET Core Web API

Understand the middleware pipeline, utilize built-in middleware components, create custom middleware for cross-cutting concerns, control middleware execution order, and apply middleware for error handling, logging, compression, and environment-specific configurations.

- **What is Middleware?** Learn how middleware handles HTTP requests and responses.
- **Built-in middleware.** Use routing, CORS, authentication, static files, and more.
- **Creating custom middleware.** Extend the pipeline with your own components.
- **Middleware pipeline order.** Understand execution order effects.
- **Exception handling middleware.** Centralize error management.
- **Registering middleware.** Configure in Program.cs properly.
- **Cross-cutting concerns.** Apply middleware for logging, compression, etc.
- **Short-circuiting middleware.** Stop pipeline processing when needed.
- **Chaining custom middleware.** Use best practices for multiple components.
- **Environment-specific middleware.** Register conditionally for environments.

Chapter 12: Exception Handling and Global Error Handling

Learn strategies for handling exceptions at controller and global levels, configure exception handling middleware, return standardized error responses, log errors for diagnostics, create custom exceptions, and extend error handling with filters.

- **Types of exceptions.** Recognize common API errors.
- **Try-catch vs global error handling.** Pros and cons of each approach.
- **Exception handling middleware.** Centralize error processing.
- **Consistent error responses.** Return standardized ProblemDetails.
- **Logging exceptions.** Capture errors for diagnostics.
- **Custom exception filters.** Extend error handling with filters.
- **Appropriate HTTP status codes.** Map exceptions to client responses.
- **Custom exception classes.** Create domain-specific exceptions.
- **Using ProblemDetailsFactory.** Customize error response details.



Chapter 13: Logging in ASP.NET Core Web API

Learn to use built-in and third-party logging providers, configure structured and contextual logging, manage log levels/categories, and implement best practices for tracking activity and diagnosing issues across your application.

- **Built-in logging providers.** Use Console, Debug, EventSource, etc.
- **Configuring logging.** Setup and customize logging providers.
- **Logging levels and categories.** Filter logs by level and category.
- **Structured logging.** Use message templates for clarity.
- **Third-party providers.** Integrate Serilog, NLog, Seq, etc.
- **Logging best practices.** Decide what and how to log efficiently.
- **Logging in controllers and middleware.** Use logging effectively across layers.

Chapter 14: Filters in ASP.NET Core Web API

Explore the purpose and types of filters, learn how to apply and create custom filters, understand filter execution order, utilize dependency injection within filters, and use filters for cross-cutting concerns like authorization, validation, and caching.

- **What are filters?** Understand interception points in request processing.
- **Types of filters.** Learn about Authorization, Resource, Action, Exception, Result filters.
- **Using built-in filters.** Apply filters appropriately.
- **Creating custom filters.** Use attributes and global registration.
- **Filter execution order.** Understand how filters are processed.
- **Real-world use cases.** Apply filters for logging, caching, validation.
- **Dependency injection in filters.** Inject services into filters.
- **Filters vs middleware.** Learn differences and when to use each.
- **Global vs scoped filters.** Choose registration strategies.
- **Short-circuiting pipeline.** Use filters for early returns.

Chapter 15: Caching in ASP.NET Core Web API

Understand caching strategies to improve performance, implement response caching and in-memory/distributed caches, manage cache expiration and invalidation, use conditional requests with ETags, and handle caching securely for sensitive data.

- **Why cache?** Improve performance and scalability with caching.
- **Caching strategies.** Learn client-side and server-side caching.
- **Response caching middleware.** Cache HTTP responses easily.
- **In-memory caching.** Use simple app-memory caching.
- **Distributed caching.** Use Redis or SQL Server for shared caches.
- **Cache profiles and busting.** Manage cache expiration and refresh.
- **Cache invalidation strategies.** Ensure data freshness.
- **Cache aside pattern.** Handle reads and writes efficiently.
- **Caching with ETags.** Use conditional requests for efficiency.
- **Security considerations.** Protect sensitive or user-specific data.

Chapter 16: Security in ASP.NET Core Web API

Learn authentication and authorization concepts, implement JWT-based security, manage refresh tokens, configure role and policy-based access control, secure endpoints, configure CORS safely, protect against common web attacks, enforce HTTPS, and apply rate limiting.

- **Authentication vs Authorization.** Understand key security concepts.
- **JWT Bearer Authentication.** Secure APIs using JSON Web Tokens.
- **Refresh Tokens.** Implement token renewal strategies.
- **Role-based and policy-based authorization.** Define access controls.
- **Securing API endpoints.** Apply authentication and authorization filters.
- **CORS configuration.** Allow cross-origin requests securely.



- **Protecting against attacks.** Mitigate CSRF, SQL Injection, XSS, etc.
- **HTTPS enforcement.** Ensure secure transport.
- **Rate limiting and throttling.** Prevent API abuse.

Chapter 17: API Versioning in ASP.NET Core Web API

Learn how to support multiple API versions and manage breaking changes. You'll configure versioning strategies (URL, query string, header), maintain backward compatibility, and handle API deprecation and documentation.

- **Why API versioning?** Manage breaking changes effectively.
- **Versioning strategies.** Use URL, query string, headers, media types.
- **Microsoft.AspNetCore.Mvc.Versioning package.** Setup and configure versioning.
- **Deprecation and backward compatibility.** Manage older API versions.

Chapter 18: Unit Testing in ASP.NET Core Web API

Learn unit testing techniques for controllers and services using xUnit, mock dependencies to isolate tests, handle async method testing, and perform integration testing without a real server.

- **Why test APIs?** Understand benefits of automated testing.
- **Testing controllers and services.** Learn unit testing approaches.
- **Using xUnit framework.** Setup test projects and tests.
- **Mocking dependencies.** Use Moq or similar to isolate tests.
- **Testing async methods.** Handle asynchronous code properly.
- **Integration testing with TestServer.** Test APIs end-to-end without real servers.

Chapter 19: Minimal API in ASP.NET Core

Learn to build lightweight and high-performance HTTP APIs using ASP.NET Core Minimal APIs. Understand how to define endpoints with minimal setup, handle routing, use dependency injection, and configure middleware in a minimal API style.

- **What are Minimal APIs?** Understand the concept and benefits of Minimal APIs in ASP.NET Core.
- **Setting up a Minimal API project.** Learn how to create and configure a Minimal API project from scratch.
- **Defining endpoints using Map methods.** Explore how to define routes with minimal code using MapGet, MapPost, etc.
- **Route parameters and request handling.** Handle input parameters and requests efficiently in Minimal APIs.
- **Dependency Injection in Minimal APIs.** Use services and DI container inside Minimal API endpoints.
- **Middleware configuration and pipeline.** Set up middleware components and understand the request pipeline.
- **Integrating with EF Core.** Connect Minimal APIs to databases using Entity Framework Core.
- **Comparing with MVC/Web API.** Understand differences and when to prefer Minimal APIs.

Angular Syllabus

Angular is a framework made by Google to build interactive web apps that run inside a web browser. It uses a language called TypeScript, which is similar to JavaScript but easier to write and understand. Angular helps developers organize their code with components and make it easier to handle things like data binding, routing, and forms. It is widely used to build modern, user-friendly websites.

Chapter 1: Introduction and Environment Setup for Angular

Develop a strong foundation in Angular by understanding its architecture, ecosystem, and core concepts. You'll set up your development environment, create and run your first Angular application, and understand the role of TypeScript and supporting tools.

- **What is Angular?** Overview and purpose of the framework.
- **Angular vs other frontend frameworks.** Unique features and strengths.
- **SPA architecture overview.** Why Angular uses a single-page app model.
- **Angular ecosystem overview.** Learn tools, libraries, and community resources.
- **Installing Node.js and Angular CLI.** Setup required tools for development.
- **Creating your first Angular application.** Step-by-step walkthrough.
- **Running and serving the app.** Development server basics.
- **Package.json.** Understand dependencies and configuration.
- **Using Angular DevTools.** Debug Angular apps effectively.

Chapter 2: Introduction to TypeScript

Build a strong foundation in TypeScript, the language behind Angular. Understand types, interfaces, classes, functions, generics, and modern ES features to write robust and maintainable code for full-stack development.

- **Introduction to TypeScript.** Learn why TypeScript improves JavaScript development with typing.
- **Basic types and type inference.** Understand built-in types and how TypeScript infers them.
- **Interfaces and type aliases.** Define contracts for objects and reusable types.
- **Functions and arrow functions.** Type your functions and use concise arrow syntax.
- **Classes and inheritance.** Implement object-oriented features with strong typing.
- **Generics for reusable code.** Use generics to build flexible, type-safe components.
- **Enums, tuples, unions, and intersections.** Explore advanced TypeScript types and combinations.
- **Advanced types.** Master mapped and conditional types for complex scenarios.
- **Modules and namespaces.** Organize code with module systems and scope control.

Chapter 3. Angular CLI, Workspace, and Project Structure

Understand Angular CLI commands and workspace organization, explore project files and folder structure, manage environment configurations.

- **What is Angular CLI?** Learn how the CLI automates common development tasks.
- **Common CLI commands.** Explore generate, serve, build, and test commands for productivity.
- **Workspace vs project.** Understand angular.json and package.json roles in configuration.
- **Folder structure explained.** Know the purpose of src/app, assets, environments folders.
- **Environment configuration files.** Manage different settings for development and production.

Chapter 4: Components and Templates in Angular

Learn the fundamentals of Angular components and templates. Build components using CLI, master template syntax, understand lifecycle hooks, implement various data binding techniques, and optimize component styling.

- **What is a component?** Understand components as the fundamental UI building blocks.
- **Creating components with CLI.** Generate components quickly and consistently.
- **Component decorator metadata.** Configure selector, template, and styles for components.
- **Templates and template syntax.** Use interpolation, property binding, and event binding.



- **Inline vs external templates and styles.** Compare advantages of both approaches.
- **Component styles and view encapsulation.** Isolate styles per component effectively.
- **Lifecycle hooks overview.** Manage `ngOnInit`, `ngOnChanges`, `ngOnDestroy` for behavior.
- **Data binding overview.** Implement one-way, two-way, and event bindings.
- **Change detection strategy.** Choose between Default and OnPush strategies.
- **Decorators: ViewChild and ContentChild.** Access child components and content projection.

Chapter 5: Directives and Pipes in Angular

Understand how directives manipulate the DOM and pipes transform data. Use built-in directives and pipes, create custom ones, and apply them effectively for dynamic UI rendering and complex data formatting.

- **What are directives?** Understand their role in DOM manipulation.
- **Built-in structural directives.** Use `*ngIf`, `*ngFor`, and `*ngSwitch` for dynamic views.
- **Attribute directives.** Modify element styles and classes dynamically with `ngClass`, `ngStyle`.
- **Creating custom directives.** Extend Angular with your own directives.
- **What are pipes?** Transform displayed data in templates efficiently.
- **Built-in pipes.** Use `DatePipe`, `CurrencyPipe`, `UpperCasePipe` for formatting.
- **Creating custom pipes.** Implement reusable data transformations.
- **Pure vs impure pipes.** Understand performance implications and behavior differences.
- **Directive input and output bindings.** Pass data to and from directives.
- **Directive lifecycle hooks.** Manage directive initialization and destruction.
- **Using async pipe for Observables.** Simplify async data handling in templates.

Chapter 6: Angular Modules and Feature Modules

Understand how Angular modules organize applications for scalability. Structure apps with root, feature, shared, and core modules, implement lazy loading, preloading strategies, and manage singleton services within modules.

- **What is an NgModule?** Learn its purpose and structure in Angular apps.
- **NgModule metadata.** Configure declarations, imports, exports, providers, bootstrap arrays.
- **Root AppModule vs feature modules.** Structure apps for scalability and reusability.
- **Creating and importing modules.** Organize code logically and effectively.
- **Shared and Core modules.** Manage reusable components and singleton services.
- **Lazy loading modules.** Improve performance by loading modules on demand.
- **Route-based splitting.** Split code by route for faster initial loading.
- **Module preloading strategies.** Balance load time and performance with preloading.
- **Using `forRoot()` and `forChild()`.** Manage singleton services across modules.
- **Singleton services in lazy-loaded modules.** Avoid multiple instances of services.

Chapter 7: Standalone Components in Angular

Learn how to build Angular applications using standalone components without NgModules. Understand the benefits, syntax, and migration strategies to standalone components for simpler and more modular Angular applications.

- **What are standalone components?** Discover the new Angular feature that reduces module dependencies.
- **Creating standalone components.** Learn the syntax and how to declare standalone components.
- **Importing dependencies directly.** Manage component dependencies without NgModules.
- **Standalone directives and pipes.** Use directives and pipes in standalone mode.

- **Bootstrapping standalone apps.** Initialize Angular applications without root NgModule.
- **Lazy loading with standalone components.** Implement lazy loading strategies in standalone architecture.
- **Migrating from NgModules.** Transition existing module-based apps to standalone components.
- **Use cases and limitations.** Understand scenarios suitable for standalone components.

Chapter 8: Reusable Component Patterns

Develop reusable, maintainable component architectures. Apply smart vs dumb component patterns, use content projection and dynamic component loading, and build scalable UI libraries and reusable dialogs.

- **Shared and reusable components.** Follow best practices for component reuse.
- **Smart (container) vs dumb (presentational) components.** Separate concerns for clarity.
- **Content projection with ng-content.** Create dynamic component templates.
- **Dynamic component loading.** Load components programmatically as needed.
- **Component libraries and design patterns.** Build UI libraries for consistency.
- **Reusable dialog and modal patterns.** Implement common dialogs efficiently.
- **Understanding @ViewChildren and @ContentChildren.** Manage child components effectively.

Chapter 9: Angular Routing and Navigation

Gain proficiency in configuring and managing routing. Handle route parameters, guards, lazy loading, navigation techniques, router events, data resolvers, and navigation error handling in single-page Angular apps.

- **Configuring Angular Router.** Setup and manage routes in your app.
- **Route parameters and query parameters.** Pass data via route URLs.
- **Child and nested routes.** Organize complex routing hierarchies.
- **Route guards: CanActivate, CanDeactivate, Resolve.** Protect routes and manage navigation.
- **Lazy loading routes.** Optimize app performance by loading routes on demand.
- **Preloading strategies.** Improve UX by preloading selected modules.
- **Navigation techniques.** Use routerLink and programmatic navigation approaches.
- **Router events and lifecycle hooks.** Respond to navigation events.
- **Route data and resolvers.** Fetch data before activating routes.
- **Handling navigation errors and fallback routes.** Manage incorrect or unauthorized navigation.

Chapter 10: Forms in Angular

Build robust forms using both template-driven and reactive approaches. Implement comprehensive validation (sync/async), manage dynamic forms, display errors, and create custom form controls for complex scenarios.

- **Template-driven forms.** Build forms using Angular's simpler approach.
- **Reactive forms.** Use FormGroup, FormControl, and FormBuilder for complex forms.
- **Form validation.** Implement built-in and custom validators.
- **Async validators.** Validate inputs asynchronously with backend calls.
- **Dynamic forms and form arrays.** Manage forms with variable controls.
- **Form submission and error handling.** Provide a good UX for users.
- **Form value and status changes observables.** React to form state changes.
- **Disabling/enabling form controls dynamically.** Manage form state programmatically.
- **Custom form controls.** Build reusable and custom form inputs.



Chapter 11: Services and Dependency Injection in Angular

Learn how to design and inject Angular services to manage business logic, share data, and control state. Configure service providers, understand DI scopes and lifecycles, and apply services for modular and maintainable code.

- **What is a service?** Understand its role in business logic and state management.
- **Creating and injecting services.** Use Angular's DI system effectively.
- **Service scope: Root, module, component.** Manage service lifetimes.
- **Hierarchical Dependency Injection.** Override providers for different scopes.
- **ProvidedIn root vs module providers.** Optimize bundle size with tree-shaking.
- **Singleton vs multiple instances.** Know when to use each pattern.
- **Using services for state management.** Centralize shared app data.
- **Injecting services into directives and pipes.** Extend service use.
- **Service lifecycles.** Manage creation and destruction in large apps.

Chapter 12: HttpClient and Communication with APIs

Gain practical skills for HTTP communication. Use HttpClient for CRUD operations, manage headers and parameters, handle errors, unsubscribe properly, implement interceptors, and support file uploads/downloads.

- **What is HttpClient?** Use Angular's HTTP communication library.
- **Making HTTP requests.** Perform GET, POST, PUT, DELETE operations.
- **Handling headers.** Manage request and response headers.
- **Working with Observables and RxJS.** Handle reactive data streams.
- **Error handling.** Implement strategies to manage HTTP errors.
- **Using HttpParams and HttpHeaders.** Pass query and header parameters.
- **Canceling requests and unsubscribing.** Prevent memory leaks.
- **Environment-specific API URLs.** Manage API endpoints per environment.
- **Using interceptors.** Add auth tokens, logging, and error handling.
- **Handling file uploads and downloads.** Manage user file operations.
- **Retry logic.** Implement automatic retries with exponential backoff.

Chapter 13: Angular Interceptors

Learn to intercept and manipulate HTTP requests and responses globally. Implement interceptors for authentication, logging, error handling, retries, and chain multiple interceptors efficiently.

- **What is an HTTP interceptor?** Understand the request/response lifecycle.
- **Purpose and use cases.** Add auth tokens, logging, and loading indicators.
- **Creating and registering interceptors.** Setup interceptors in Angular.
- **Request and response interception.** Modify and handle HTTP data.
- **Global error handling.** Centralize HTTP error management.
- **Logging HTTP traffic.** Monitor API calls easily.
- **Chaining multiple interceptors.** Order interceptors for complex flows.
- **Retry logic.** Implement retry policies for HTTP requests.

Chapter 14: Component Interaction and Event Handling

Master communication between components using inputs, outputs, event emitters, shared services, RxJS Subjects, and direct DOM or custom events—enabling robust, decoupled component architectures.

- **Parent to child communication.** Use input properties for data passing.
- **Child to parent communication.** Emit events with EventEmitter.
- **Cross-component communication.** Use RxJS Subjects and shared services.
- **Handling DOM events.** Bind and propagate native events.
- **ViewChild and ContentChild queries.** Access child components and templates.
- **Service-based event communication.** Share events across components.
- **Handling custom and native events.** Combine event handling approaches.



Chapter 15: Advanced RxJS and Reactive Programming in Angular

Deepen your knowledge of RxJS to build powerful reactive Angular applications. Use observables and subjects, apply common operators, handle errors, manage subscriptions, and create custom reactive logic.

- **What is RxJS?** Learn reactive programming fundamentals.
- **Core concepts.** Understand Observables, Subjects, BehaviorSubjects, ReplaySubjects.
- **Hot vs cold observables.** Know the difference in data streams.
- **Common operators.** Use map, filter, switchMap, mergeMap, debounceTime effectively.
- **Error handling and retry logic.** Manage errors in streams gracefully.
- **Subscription management.** Avoid memory leaks with proper unsubscribing.
- **Creating custom operators.** Build reusable reactive utilities.
- **Higher-order Observables.** Handle complex nested streams.

Chapter 16: Angular Signals

Explore Angular's reactive state management using Signals for efficient and fine-grained reactivity in Angular applications. Understand how Signals differ from Observables and how to use them in components and services.

- **Introduction to Angular Signals.** Understand the motivation and basic concepts behind Signals.
- **Creating and updating signals.** Learn to declare signals and update reactive state.
- **Signal effects and computed signals.** Use side effects and derived signals for reactive programming.
- **Using signals in templates.** Bind signals directly in Angular templates for automatic updates.
- **Comparing with RxJS Observables.** Learn differences and when to use Signals or Observables.
- **State management patterns.** Implement app-wide state using Signals efficiently.
- **Integrating with existing apps.** Combine Signals with current Angular codebases.
- **Performance and best practices.** Optimize reactive code using Signals.

Chapter 17: State Management with NgRx (Redux Pattern)

Implement large-scale state management using NgRx. Learn the Redux pattern, manage state with actions/reducers/effects/selectors, debug with devtools, manage entities, and migrate to global state as apps scale.

- **Why state management?** Manage app complexity and data flow.
- **Introduction to NgRx and Redux.** Learn concepts and terminology.
- **Actions, reducers, selectors.** Core building blocks of NgRx.
- **Effects.** Handle side effects and asynchronous operations.
- **Store devtools.** Debug state changes easily.
- **Entity state management.** Manage collections efficiently.
- **Router integration.** Sync state with navigation.
- **Migration strategies.** Move from local state to global state management.
- **Feature state modules and lazy loading.** Organize large app states.
- **Using createAction and createReducer APIs.** Modern NgRx syntax.

Chapter 18: Error Handling and Global Error Management

Implement robust error handling strategies. Centralize error logic, provide user-friendly feedback, integrate error logging, implement retry and fallback mechanisms, and use Angular's ErrorHandler class.

- **What are error boundaries?** Capture and isolate errors.
- **Handling errors in templates, components, and services.** Apply best practices.
- **Centralized error handling strategy.** Use a global error handler class.
- **Displaying user-friendly messages.** Improve UX with clear feedback.



- **Logging errors.** Send error data to backend or monitoring tools.
- **Integration with HTTP interceptors.** Handle HTTP errors globally.
- **Retry and fallback mechanisms.** Ensure app stability on failures.
- **Using Angular's ErrorHandler class.** Customize global error processing.
- **Reporting errors to external services.** Integrate with monitoring platforms.

Chapter 19: Angular Material & UI Component Libraries

Get hands-on experience with Angular Material and other UI libraries. Install, configure, theme, customize components, ensure accessibility, and integrate with various design systems.

- **What is Angular Material?** Explore a comprehensive UI component library.
- **Installing and configuring Material.** Setup and customize your app theme.
- **Using Material components.** Use buttons, forms, dialogs, tables, navigation.
- **Theming and custom styling.** Apply custom looks with theming support.
- **Other popular UI libraries.** Learn about PrimeNG, Bootstrap alternatives.
- **Customizing Material themes.** Modify palettes and styles.

Chapter 20: Syncfusion UI Components and Integration in Angular

Learn to integrate Syncfusion UI components into Angular apps. Handle installation, data binding, events, localization, licensing, performance tuning, and NgRx integration.

- **Overview of Syncfusion suite.** Discover key Angular components available.
- **Installing Syncfusion packages.** Set up components in your project.
- **Using key components.** Work with DataGrid, Charts, Scheduler, Rich Text Editor.
- **Data binding and event handling.** Manage Syncfusion control data and events.
- **Customizing components.** Adjust styles and behaviors.
- **Licensing and production considerations.** Comply with Syncfusion licensing.
- **Performance tuning.** Optimize UI responsiveness.
- **Localization and globalization.** Adapt components for different locales.
- **NgRx integration.** Manage Syncfusion state with NgRx.

Chapter 21: File Uploads and Downloads

Develop feature-rich file upload/download solutions. Build secure, user-friendly UIs, validate and preview files, support chunked uploads, manage progress, and ensure robust security.

- **Implementing file upload UI.** Create intuitive file input controls.
- **Validations.** Enforce file size, type, and security restrictions.
- **Uploading files to backend APIs.** Use HttpClient to send files.
- **Handling multiple files and progress bars.** Improve UX with feedback.
- **Downloading files.** Trigger client-side downloads.
- **Security considerations.** Prevent malicious file uploads.
- **Drag-and-drop file upload.** Support user-friendly drag-and-drop features.
- **Displaying file previews.** Show file previews before upload.
- **Chunked uploads.** Handle large files by uploading in parts.

Chapter 22: Angular Animations

Explore Angular's animation framework. Create triggers, states, and transitions; animate routes and lists; implement callbacks; conditionally animate UI elements; and optimize animation performance.

- **Basics of Angular animations.** Use the @angular/animations package.
- **Animation triggers, states, and transitions.** Define animation behaviors.
- **Route transition animations.** Animate page navigation smoothly.
- **Animating lists and UI elements.** Enhance interactivity with animations.
- **Staggered and sequential animations.** Create complex animation sequences.
- **Animation callbacks and event listeners.** React to animation events.
- **Performance optimization.** Ensure animations run smoothly on all devices.



Chapter 23: Security in Angular Applications

Strengthen Angular app security by preventing XSS, sanitizing inputs, protecting routes, managing JWTs and storage securely, role-based UI rendering, and implementing content security policies.

- **Preventing Cross-Site Scripting (XSS).** Use sanitization and safe coding.
- **Sanitizing user inputs.** Apply Angular's built-in sanitization mechanisms.
- **Securing route access with guards.** Control user navigation securely.
- **Managing authentication tokens (JWT).** Store and use tokens safely.
- **Securing local and session storage.** Protect sensitive client data.
- **Role-based UI rendering.** Show/hide UI elements based on user roles.
- **Content Security Policy (CSP).** Enforce policies to reduce attacks.

Chapter 24: Performance Optimization in Angular

Optimize Angular app speed and efficiency using AOT compilation, change detection strategies, lazy loading, bundle optimization, asset loading improvements, DevTools profiling, and Web Workers.

- **Ahead-of-Time (AOT) compilation.** Reduce bundle size and speed startup.
- **Change detection strategies.** Optimize Angular's change detection cycle.
- **Lazy loading modules and components.** Load code on demand.
- **Bundle size optimization.** Remove unused code with tree shaking.
- **Optimizing image and asset loading.** Improve page load speed.
- **Performance profiling with Angular DevTools.** Identify bottlenecks easily.
- **Preloading strategies and manual chunking.** Balance load and performance.
- **Using Web Workers.** Offload heavy processing from the main thread.

Chapter 25: Angular Internationalization (i18n) and Localization

Prepare apps for a global audience by marking translatable content, extracting and managing translation files, formatting locale-specific data, enabling runtime language switching, and using third-party i18n libraries.

- **What is Angular i18n?** Understand the benefits of multi-language apps.
- **Marking templates and strings.** Prepare text for translation.
- **Extracting and merging translation files.** Manage translations effectively.
- **Locale-specific formatting.** Format dates, numbers, currencies per locale.
- **Runtime language switching.** Enable users to change languages on the fly.

Chapter 26: Progressive Web Apps (PWA) with Angular

Enable advanced web features by building Progressive Web Apps. Implement service workers, caching, offline capabilities, background sync, manifest customization, and push notifications for modern web experiences.

- **What is a PWA?** Learn key features and advantages.
- **Adding PWA support.** Use Angular CLI to quickly enable PWA capabilities.
- **Service workers and caching strategies.** Provide offline functionality.
- **Background sync.** Keep data updated when back online.
- **Manifest and app icons.** Customize app appearance on devices.

Chapter 27: Unit Testing in Angular

Master unit testing Angular components, services, and pipes using Jasmine and Karma. Learn to write effective test cases, use test doubles, handle asynchronous code.

- **Importance of unit testing.** Understand why testing is essential for maintainable Angular apps.
- **Setting up Jasmine and Karma.** Configure the testing environment and tools.
- **Writing test cases.** Develop test suites for components, services, and directives.
- **Using TestBed.** Use Angular's TestBed to create testing modules and components.
- **Mocking dependencies and services.** Isolate units by mocking external dependencies.
- **Testing asynchronous code.** Handle async operations with fakeAsync and async utilities.



Chapter 28. Reporting in ASP.NET Core Web API and Angular

Learn how to generate, export, and display reports efficiently using ASP.NET Core Web API and Angular. Explore PDF and Excel generation, interactive dashboards, and secure delivery of reports to users.

- **Introduction to Reporting.** Understand why reporting is essential and common report types in web apps.
- **Gathering reporting requirements.** Identify data sources and user needs for meaningful reports.
- **Generating reports in ASP.NET Core Web API.** Use libraries like iText7 and ClosedXML for PDF and Excel.
- **Creating PDF reports.** Design styled, password-protected, and multi-page PDF documents.
- **Exporting Excel reports.** Generate formatted Excel files with multiple sheets and formulas.
- **Designing report APIs.** Build RESTful endpoints that serve reports efficiently and securely.
- **Server-side report rendering.** Render HTML or other formats for complex reports.
- **Consuming reports in Angular.** Display reports using PDF viewers, data grids, and export buttons.
- **Interactive dashboards and visualizations.** Integrate charts using Chart.js, Syncfusion, or similar libraries.
- **Downloading and streaming reports.** Handle secure and performant file downloads from the frontend.
- **Report caching and optimization.** Implement caching to improve report generation performance.
- **Scheduling and automated report delivery.** (Optional) Automate report generation and emailing via background jobs.

Why .NET Full Stack using ASP.NET Core Web API and Angular?

.NET Full Stack development using ASP.NET Core Web API and Angular is a popular choice because it allows developers to build complete web applications using powerful, modern technologies that work well together. ASP.NET Core Web API provides a fast and secure backend to handle data and business logic, while Angular creates dynamic and responsive user interfaces on the front end. Both technologies support cross-platform development, meaning the apps can run on different operating systems. Using this combination helps streamline development, improve performance, and deliver a great user experience with a single, consistent technology stack.

Follow Us on Social Media

 Subscribe to our YouTube Channel: <https://www.youtube.com/@DotNetTutorials/playlists>

 Join Our Facebook Group: <https://www.facebook.com/groups/501905403704561>

 Like & Follow Our Facebook Page: <https://www.facebook.com/tutorialsdotnet>

 Join Our LinkedIn Group: <https://www.linkedin.com/groups/14539101/>

 Follow Our LinkedIn Company Page: <https://www.linkedin.com/company/dot-net-tutorials/>